



ORBIS: AN INTELLIGENT ACADEMIC SUPPORT SYSTEM WITH AI AND RAG

by

ATAKAN GÜL, 121200152
ARDA KAAAN YILDIZ, 122200045

Supervised by

ÖZGÜR ÖZDEMİR

Submitted to the
Faculty of Engineering and Natural Sciences
in partial fulfillment of the requirements for the

Bachelor of Science

in the
Department of Computer Engineering

June, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	ii
1 Realistic Constraints	1
1.1 Social, Environmental and Economic Impact	1
1.2 Cost and Time Analysis	2
1.3 Standards	3
1.3.1 NSPE Code Of Ethics	3
1.3.2 IEEE Standards	4
2 Risk Analysis And Changes	4
2.1 Challenges	5
2.2 Weak And Strong Points	6
2.2.1 Weak Points	6
2.2.2 Strong Points	6
3 Contributions	7
4 Conclusion	7
References	9

1 Realistic Constraints

This report discusses the realistic constraints of Orbis. Report 1 already explains the technical architecture, methodology, experiments and results. Therefore, this document focuses on the social, environmental, economic, ethical, standardization, risk and contribution parts of the project.

Orbis is an academic support system that uses AI and Retrieval-Augmented Generation to detect academic obligations, match them with student profiles and help students follow university regulations. Since the project is related to education and student data, it must be evaluated not only as a software product but also as an engineering system that can affect real users.

1.1 Social, Environmental and Economic Impact

For an engineering project to be successful, it should create a useful effect for people or institutions. Orbis mainly creates a social impact by helping students understand academic rules earlier and more clearly. University regulations can be long, scattered and difficult to follow. Students may miss deadlines not because they are careless, but because they do not know which regulation applies to them. Orbis reduces this problem by turning official regulation text into personalized, source-backed academic tasks.

This social impact is related to the United Nations Sustainable Development Goal 4, Quality Education, and Goal 10, Reduced Inequalities [1]. Students with less advising experience, double-major students, scholarship students, internship students and students near graduation can benefit from earlier warnings. However, wrong advice can also harm students. For this reason, Orbis should not replace advisors or Student Affairs. It should work as a support tool that shows the source document and allows human review when needed.

The environmental impact is smaller but still positive. A proactive digital advising system can reduce the need for printed handbooks, physical notices and repeated manual document distribution. The system also supports sustainability by caching embeddings and re-processing changed regulations only when needed. This avoids unnecessary repeated AI calls and keeps the computational cost lower.

Economically, Orbis can reduce repeated advising workload. In the evaluated run, the event pipeline processed 80 regulation sources, 935 regulation chunks and 323 candidate rules in 59 minutes, finally accepting 178 actionable obli-

gations. Doing the same filtering manually would require much more human time. The project therefore has an economic benefit for the university by helping advisors and administrative units focus on more complex cases.

1.2 Cost and Time Analysis

The main cost of Orbis is labor. The software stack is mostly open source and the project was developed with existing student hardware. Still, the time spent on research, design, implementation, testing and reporting should be counted as a real engineering cost.

For this analysis, the hourly engineering labor cost is estimated as 300 TL/hour.

- Research, requirement analysis and planning: approximately 120 hours, 36,000 TL.
- Database design, architecture and data model work: approximately 110 hours, 33,000 TL.
- Knowledge-base crawling, taxonomy generation and RAG preparation: approximately 160 hours, 48,000 TL.
- Event pipeline, rule extraction and assignment matching: approximately 150 hours, 45,000 TL.
- Submission agent, appeal flow and safety tests: approximately 90 hours, 27,000 TL.
- Frontend/backend integration, debugging and user workflows: approximately 90 hours, 27,000 TL.
- Evaluation and reporting: approximately 150 hours, 45,000 TL.

The total labor estimate is approximately 870 hours. With 300 TL/hour, the labor cost becomes approximately 261,000 TL.

The technical cash cost is much lower:

- FastAPI, PostgreSQL, `pgvector`, React and Python libraries: 0 TL, because they are open-source tools.
- Existing development laptops: 0 TL additional project cost.

- Measured LLM evaluation cost: approximately 60 TL.
- Estimated cloud hosting for a small deployment: approximately 2,500 TL/month.
- Estimated three-month hosting cost: approximately 7,500 TL.
- Electricity and local development overhead: approximately 750 TL.

The approximate total research and development cost, including labor and technical costs, is therefore around 269,310 TL. If Orbis becomes a production system, the main continuing cost would be server hosting and maintenance rather than LLM usage.

1.3 Standards

In this section, the standards of NSPE Code of Ethics, IEEE software engineering standards and relevant data-protection/accessibility regulations are discussed.

1.3.1 NSPE Code Of Ethics

The NSPE Code of Ethics states that engineers should hold public safety, health and welfare paramount [2]. In Orbis, the main public welfare issue is student academic welfare. An incorrect notification or wrong submission decision can affect a student’s academic progress.

For this reason, Orbis follows these ethical principles:

- Engineers shall avoid deceptive acts.
- Engineers shall acknowledge errors and should not hide known limitations.
- Engineers should act responsibly, ethically and lawfully.
- Systems that affect users should be explainable and reviewable.

In the project, these principles were applied by keeping source URLs, evidence text, audit logs and human appeal paths. The report also states the weak points of the system instead of presenting the prototype as fully complete.

1.3.2 IEEE Standards

The following IEEE/ISO/IEC standards are related to Orbis:

- **IEEE/ISO/IEC 16085-2020 – Risk Management.** This standard is relevant because Orbis has risks such as wrong academic advice, missed obligations, stale regulations, privacy leakage and prompt injection [3].
- **ISO/IEC/IEEE 12207 – Software Life Cycle Processes.** This standard is related to the project’s software life cycle: requirements, design, implementation, testing, evaluation and maintenance [4].
- **ISO/IEC/IEEE 29148-2018 – Requirements Engineering.** Orbis converts student needs and university rules into explicit software requirements and atomic submission checks [5].
- **ISO/IEC/IEEE 24748-7000-2022 – Ethical Concerns During System Design.** This standard is relevant because Orbis uses AI in an educational decision-support setting [6].

In addition to IEEE standards, the project should follow data protection and accessibility principles. GDPR is relevant for European data-protection expectations [7], and KVKK Law No. 6698 is relevant for personal data processing in Turkey [8]. Since Orbis can process GPA status, scholarship information, internship status and course enrollment, only necessary student data should be used. The interface should also follow WCAG 2.2 accessibility principles so that academic support is usable by different students [9].

2 Risk Analysis And Changes

In this section, the main project risks and the changes or backup plans for these risks are explained.

- **Risk 1: Incorrect academic advice.** The assignment matcher can assign a regulation to the wrong student profile. Change: every assignment should include source evidence and should be dismissible or reviewable. B plan: high-impact assignments should be routed to an advisor before becoming final.

- **Risk 2: Missed obligations.** A real obligation may not be assigned to a student. Change: the full source document remains searchable through RAG even if the event pipeline misses it. B plan: add a recall-oriented second pass and advisor escalation for disputed cases.
- **Risk 3: Prompt injection in submitted documents.** A submitted file may contain text that tries to manipulate the LLM. Change: the submission agent first decomposes assignment requirements and then checks the file against those requirements. B plan: suspicious or low-confidence cases should go directly to human review.
- **Risk 4: Stale regulations.** University rules can change after the first crawl. Change: store source timestamps and content hashes. B plan: schedule re-crawls and keep semester-specific regulation snapshots.
- **Risk 5: Privacy leakage.** Student profiles contain sensitive academic information. Change: send only necessary fields to the model and log outbound prompts. B plan: use an on-premise model or anonymized profiles for sensitive cases.
- **Risk 6: SIS integration delay.** Direct integration with the official Student Information System may not be immediately available. Change: use synthetic profiles and manual CSV/profile import during the prototype stage. B plan: keep Orbis as an advisory layer until official integration is approved.

Change management should be handled carefully. Regulation changes should trigger re-crawling and re-extraction. Prompt or model changes should be tested on the stored evaluation set before being used. Database changes should use migrations and backups. If a new model or prompt performs worse, the system should be rolled back to the previous stable version.

2.1 Challenges

In this section we can summarize the challenges faced during the project:

- **Data quality:** University web pages include regulations, announcements, event pages and duplicated bilingual content. Separating binding regulations from general information was difficult.

- **Rule ambiguity:** Some regulation sentences are formal but not directly assignable to a student. The system had to reject broad or non-actionable text.
- **Context matching:** Some obligations depend on GPA, internship state, scholarship state or graduation year. These cases are harder than simple keyword matching.
- **Evaluation:** There was no ready benchmark for Istanbul Bilgi University regulations. The gold labels had to be prepared manually.
- **Security:** The submission agent had to be tested against prompt-injection attempts inside uploaded documents.
- **Integration:** A full production version would require official SIS integration and institutional permission.

2.2 Weak And Strong Points

2.2.1 Weak Points

- The assignment-matching stage is the weakest measured subsystem. Its overall F_1 score is 0.51, which is acceptable for a prototype but not enough for unsupervised production decisions.
- The submission-agent test set is small. It includes 32 deterministic cases, so the result should not be interpreted as a complete security guarantee.
- The system currently uses synthetic or manually seeded student profiles instead of live SIS data.
- Stale regulations remain a risk unless scheduled re-crawling and regulation snapshots are used.
- Privacy and legal compliance would require a formal university deployment process.

2.2.2 Strong Points

- Orbis is traceable. Assignments and decisions are connected to source URLs, evidence text or reasoning logs.

- The event extraction stage has high precision. It reached 0.978 precision against the gold ground truth.
- The system is modular. The crawler, taxonomy system, event orchestrator, matcher, submission agent and frontend can be improved separately.
- The LLM cost is low compared to labor and hosting costs.
- The submission agent includes an appeal path, so rejected students can flag a case for human review.

3 Contributions

- **Author 1, Atakan Gül:** mainly worked on the EventPipelineOrchestrator, LLM reasoning agents, proactive event extraction, assignment matching, FastAPI backend integration, submission-review agent and safety/evaluation scripts.
- **Author 2, Arda Kaan Yıldız:** mainly worked on the Retrieval-Augmented Generation and taxonomy side of the project, including the institutional knowledge base, semantic discovery pipeline, URL/taxonomy classification and PostgreSQL/pgvector database architecture.
- **Supervisor, Özgür Özdemir:** supervised the project, gave feedback during the development process and guided the technical and reporting parts of the study.

Both authors also contributed to planning, debugging, manual labeling, evaluation, presentation preparation and report writing.

4 Conclusion

Orbis is an AI-supported academic advising project that aims to make university regulations more visible, personalized and actionable. The project has positive social and economic effects because it can reduce student confusion and administrative workload. It also supports sustainability by reducing repeated manual document handling and by using cached embeddings and controlled model calls.

The project also has important risks. Wrong academic advice, missed obligations, stale regulations, privacy leakage and prompt injection must be handled carefully. For this reason, Orbis should be used as an advisory and evidence-based support system, not as a final decision authority. Human review, source evidence, appeal paths and change management are necessary parts of the design.

As a result, the project follows the required engineering constraints, standards, risk analysis and contribution expectations. With stronger SIS integration, larger evaluation sets and formal deployment controls, Orbis can become a sustainable academic support tool for the university.

References

- [1] United Nations, “Transforming our world: The 2030 agenda for sustainable development.” <https://sdgs.un.org/2030agenda>, 2015.
- [2] National Society of Professional Engineers, “Nspe code of ethics for engineers.” <https://www.nspe.org/resources/ethics/code-ethics>, 2026.
- [3] IEEE Standards Association, “IEEE/ISO/IEC 16085-2020: Systems and software engineering – life cycle processes – risk management.” <https://standards.ieee.org/ieee/16085/6932/>, 2020.
- [4] IEEE Standards Association, “IEEE/ISO/IEC 12207-2017: Systems and software engineering – software life cycle processes.” <https://standards.ieee.org/ieee/12207/5672/>, 2017.
- [5] IEEE Standards Association, “IEEE/ISO/IEC 29148-2018: Systems and software engineering – life cycle processes – requirements engineering.” <https://standards.ieee.org/standard/29148-2018.html>, 2018.
- [6] IEEE Standards Association, “IEEE/ISO/IEC 24748-7000-2022: Systems and software engineering – life cycle management – standard model process for addressing ethical concerns during system design.” <https://standards.ieee.org/>, 2022.
- [7] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the european parliament and of the council.” <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>, 2016.
- [8] Kişisel Verileri Koruma Kurumu, “Personal data protection law no. 6698.” <https://www.kvkk.gov.tr/Icerik/6649/Personal-Data-Protection-Law>, 2016.
- [9] World Wide Web Consortium, “Web content accessibility guidelines (WCAG) 2.2.” <https://www.w3.org/TR/WCAG22/>, 2023.